WPI

CS 534
Artificial Intelligence

Week 9: Natural Language Processing I

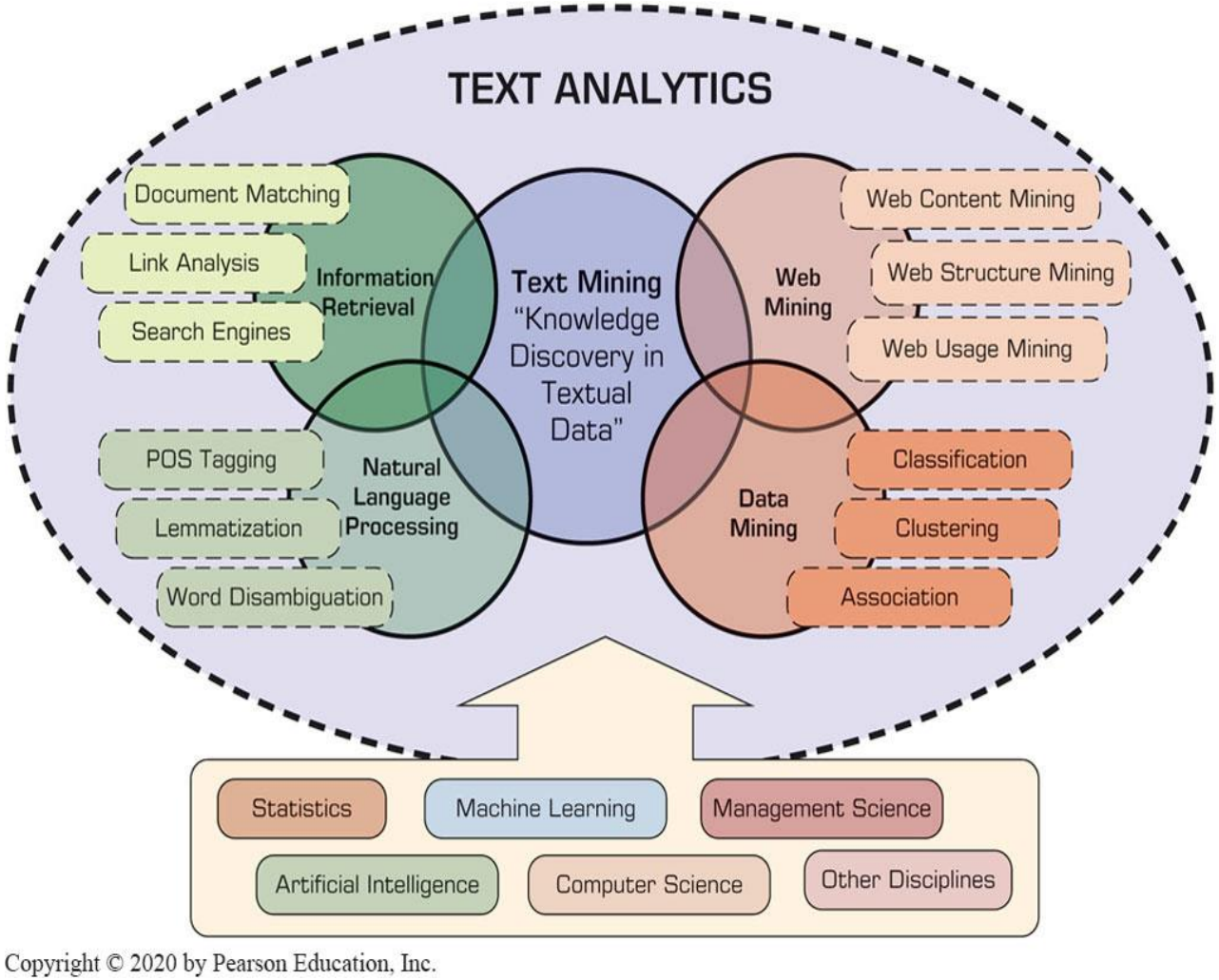
By

Ben C.K. Ngan

Worcester Polytechnic Institute

Text Analytics and Text Mining Overview

- Text analytics is a broader concept that includes information retrieval, data mining, text mining, web mining, and NLP.
- Text mining is to turn unstructured textual data into actionable information through the application of NLP and analytics.
- Statistics, ML, AI, CS, and Management Science, and other skill disciplines are used to develop the applications in those areas.



Text Mining

- **Data mining** is the process of identifying patterns in the data stored in **structured databases** where the data are organized in records structured by categorical, ordinal, and continuous variables.

- **Text mining** is the process of extracting patterns, i.e., useful information and knowledge, from large amounts of **unstructured data sources** that include a collection of unstructured or less structured data files, e.g., word documents, PDF files, text excerpts, and XML files.

- **Most Popular Applications**:
  - Information Extraction: Identify key phrases and their relationships within text
  - Topic Modeling: Identify the documents that their topics may be interested by users.
  - Summarization: Summarize a document into a synopsis.
  - Categorization: Classify text based upon a predefined set of categories
  - Clustering: Group similar documents without a predefined set of categories
  - Question & Answering: Find the best answer to a given question
  - And More!

Text Mining Terminology

- **Unstructured data**: Unstructured data do not have a predetermined format and are stored in the form of textual documents.

- **Corpus**: A large and structured set of texts prepared for the purpose of conducting knowledge discovery.

- **Terms**: A term is a single word or multiword phrase extracted directly from the corpus of a specific domain by means of NLP methods.

- **Concepts**: Concepts are features generated from a collection of documents. Concepts are the result of higher-level abstraction.

- **Lemmatization**: Lemmatization is the process of reducing inflected words to their base form. For instance, studies, studied, and studying are all based on the base form, study.

Text Mining Terminology

- **Stop words**: Stop words are words that are filtered out prior to or after processing natural language data (i.e., text). A list includes ***articles (a, an, the)***, ***prepositions (of, on, for)***, ***auxiliary verbs (is, are, was, were)***, and context-specific words that are deemed not to have differentiating value.

- **Synonyms and Polysemes**:
  - ***Synonyms*** are syntactically different words (i.e., spelled differently) with identical or at least similar meanings (**e.g., movie, film, and motion picture**).
  - ***Polysemes***, which are also called homonyms, are syntactically identical words (i.e., spelled exactly the same) with different meanings (**e.g., bow can mean "to bend forward," "the front of the ship," "the weapon that shoots arrows," or "a kind of tied ribbon"**).

- **Tokenizing**: A token is a categorized block of text in a sentence. The block of text corresponding to the token is categorized according to the function it performs. This assignment of meaning to blocks of text is known as tokenizing. ***Each token may be a single word or a multi-words***.

- **Term Dictionary**: This is a collection of ***terms specific to a narrow field*** that can be used to restrict the extracted terms within a corpus.

- **Word Frequency**: This is the ***number of times*** a word is found in a specific document.

Text Mining Terminology

- **Part-of-Speech Tagging**: This is the process of marking the words in a text as corresponding to a particular part of speech (***e.g., nouns, verbs, adjectives, adverbs, etc.***) based on a word's definition and the context in which it is used.

- **Morphology**: This is the branch of the field of linguistics and a part of NLP that ***studies the internal structure of words***, how they are formed, and their relationship to other words in the same language.

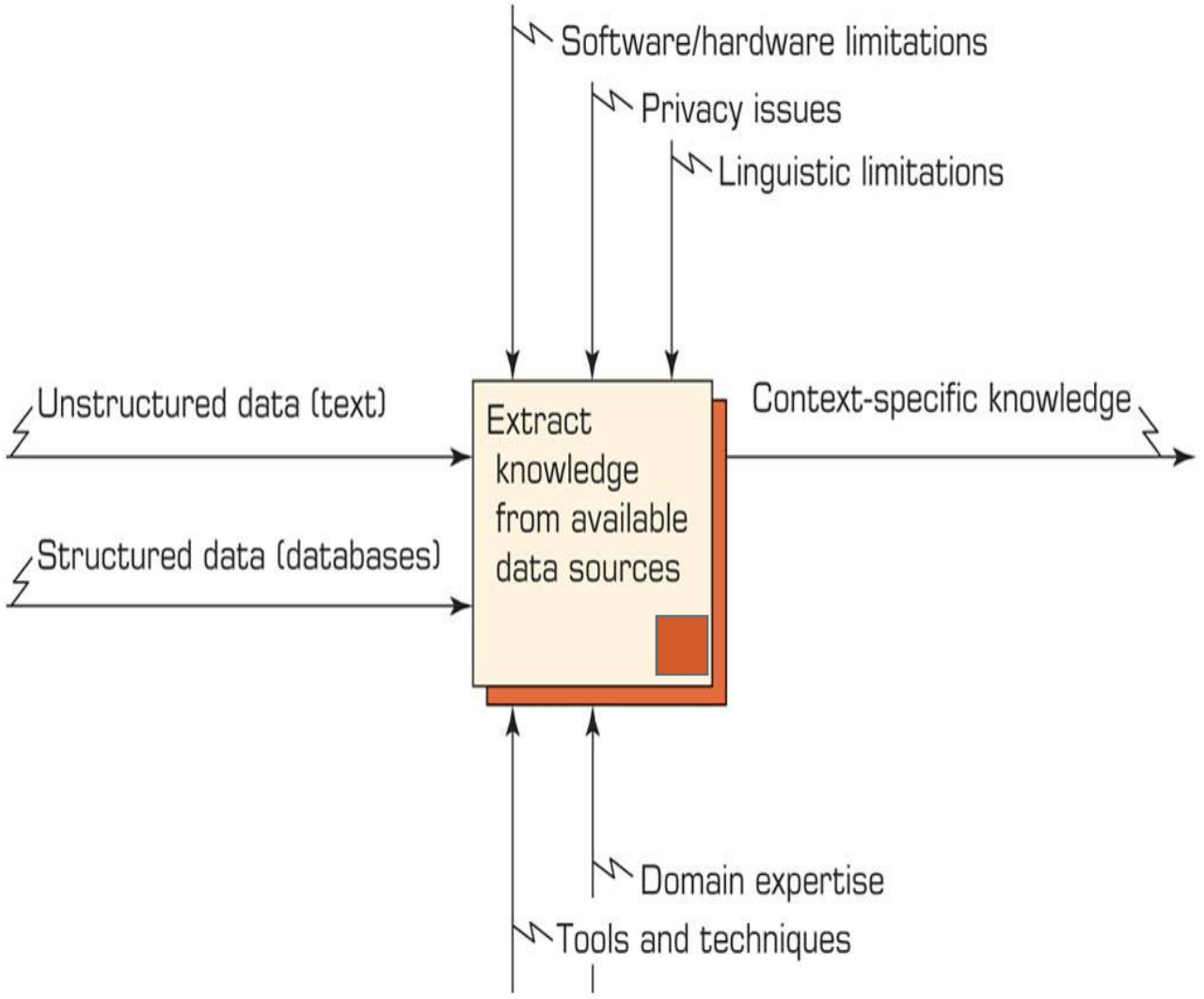
- **Term-by-document Matrix**: It is the representation schema of the frequency-based relationship between the terms and documents in ***tabular format*** where terms are listed in columns, documents are listed in rows, and ***the frequency between the terms and documents*** is listed in cells as integer values.

- **Singular Value Decomposition**: This dimensionality reduction method is used to transform the term-by-document matrix to a manageable size by generating an intermediate representation of the frequencies using a matrix manipulation method, similar to principal component analysis.

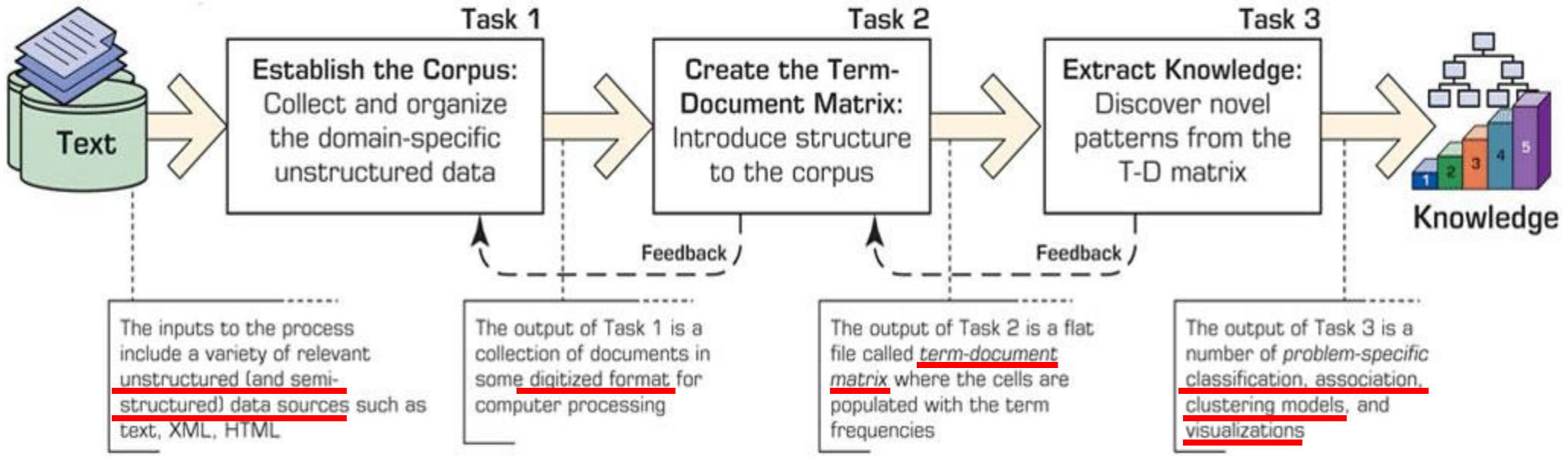
Text Mining (TM) Process

- **Input (Left)**: Unstructured and Structured Data
- **Constraint (Top)**: Software/Hardware Limitations, Privacy Issues, and Linguistic Limitations
- **Mechanism (Bottom)**: Proper Techniques, Software Tools, and Domain Expertise
- **Output (Right)**: Context-specific knowledge

The text mining process is to use both unstructured (textual) data along with structured data if relevant to the problem being addressed and available ***to extract meaningful and actionable patterns for better decision making.***



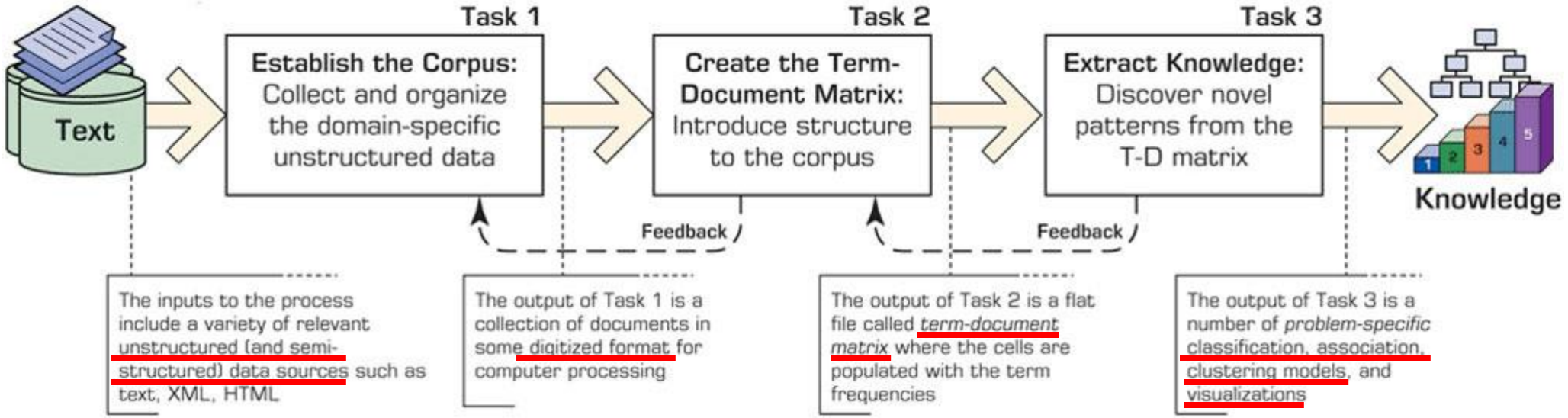
TM Process – Task 1: Corpus Establishment



- **Collect all the documents** related to the context (domain of interest) being studied. This collection may include ***textual documents, XML files, e-mails, Web pages, and short notes***.

- The text documents are then **transformed and organized** in a manner such that they are all in the same representational form (e.g., ***ASCII text files***) for computer processing.

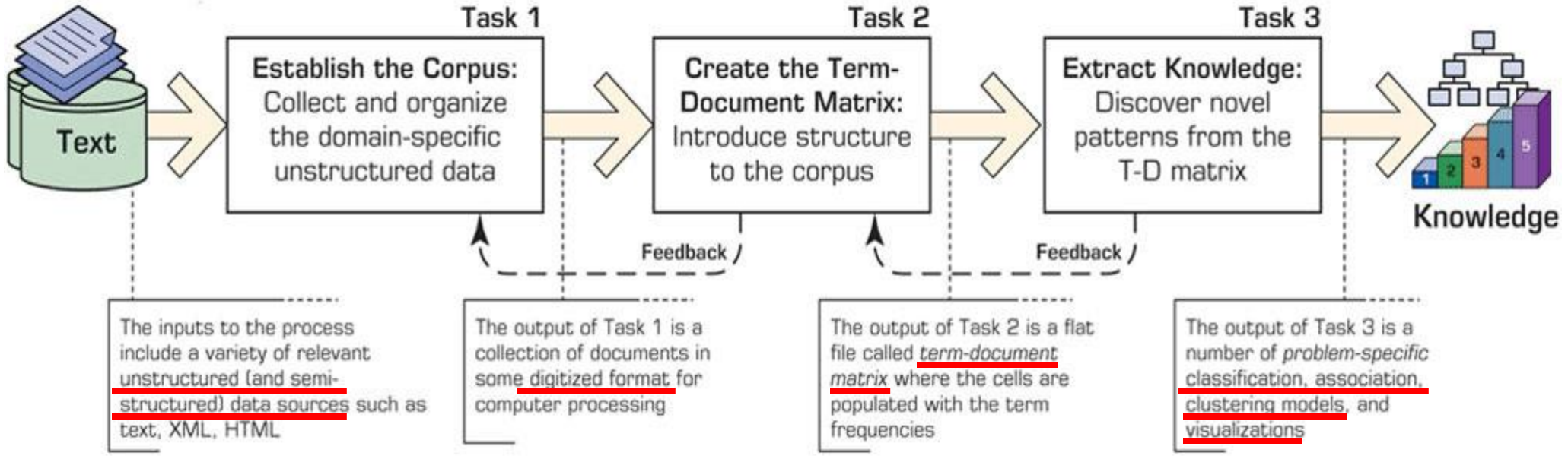
TM Process – Task 2: Term-Document Matrix



- The corpus are used to **create the term–document matrix (TDM)**.
- In the TDM, the rows represent the documents and the columns represent the terms.
- The relationships between the terms and documents are characterized by ***indices*** (e.g., the number of term occurrences).

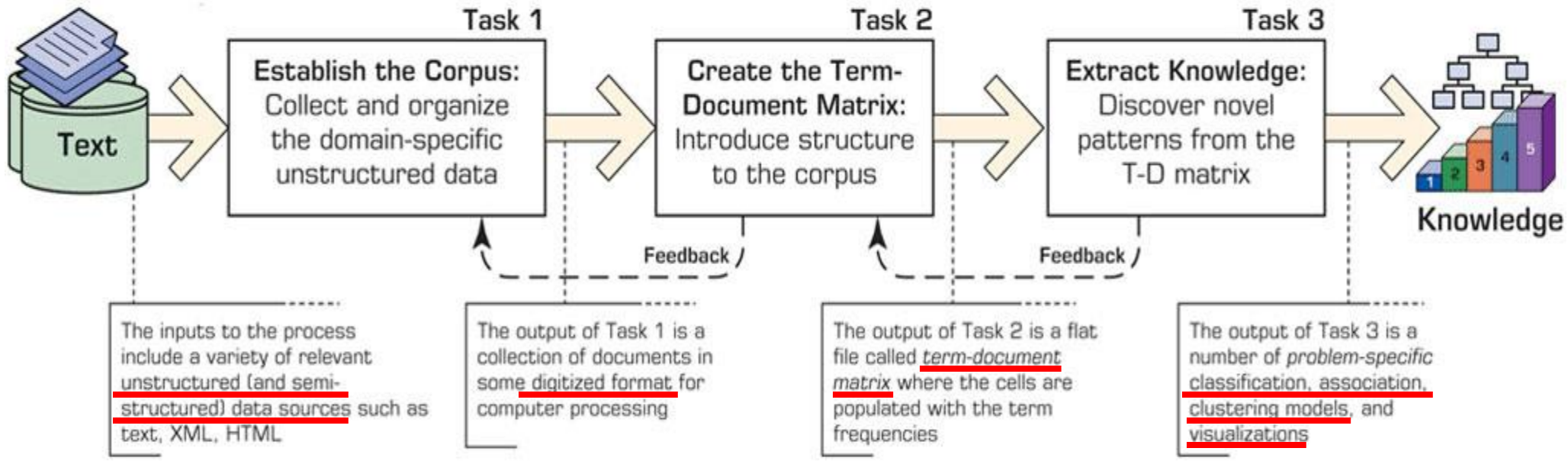
Documents \ Terms	Investment Risk	Project Management	Software Engineering	Development	SAP	…	
Document 1	1			1			
Document 2		1					
Document 3			3		1		
Document 4		1					
Document 5			2	1			
Document 6	1			1			
…							

TM Process – Task 2: Term-Document Matrix



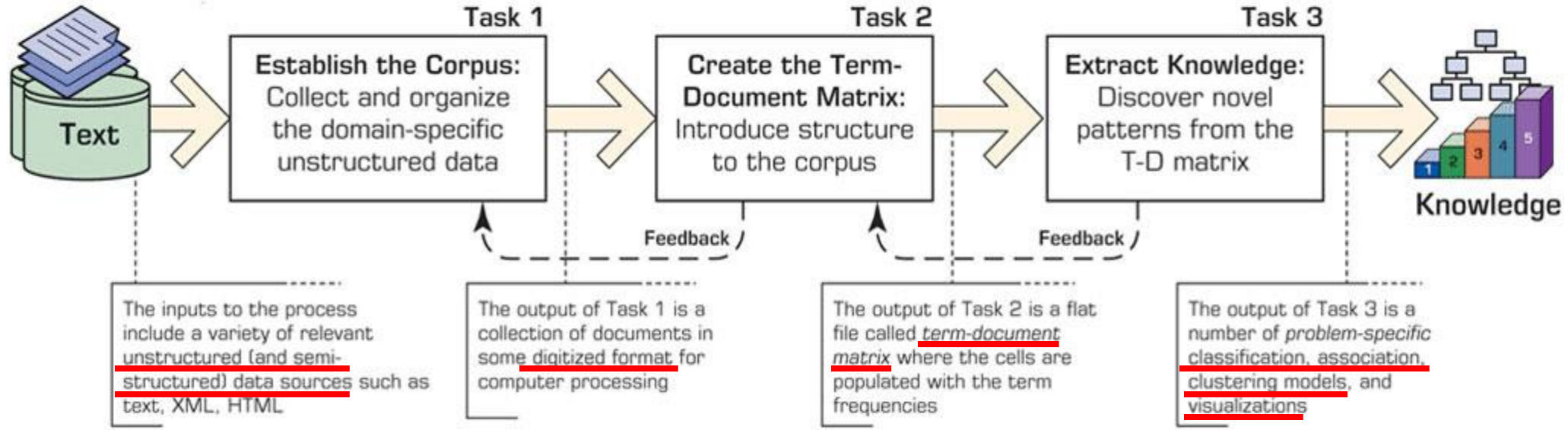
- If the corpus includes a rather large number of documents, the TDM will have a very large number of terms, i.e., **a sparse matrix**.
  - Processing such a large matrix might be ***time consuming***
  - It may lead to extraction of ***inaccurate patterns***.
- (1) What is the best representation of the indices?
- (2) How can we reduce the dimensionality of this matrix to a manageable size?

TM Process – Task 2: Term-Document Matrix



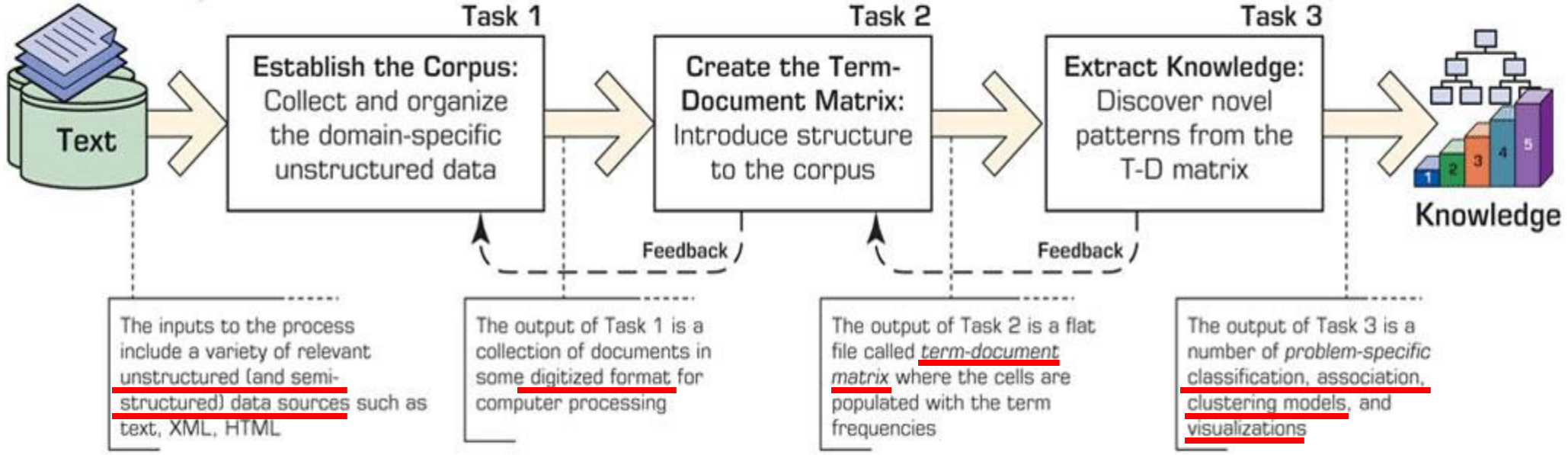
- (1) What is the best representation of the indices?
  - Term Frequency
  - Binary Frequency
  - Term Frequency-Inverse Document Frequency (TF-IDF)
  - And More

TM Process – Task 2: Term-Document Matrix



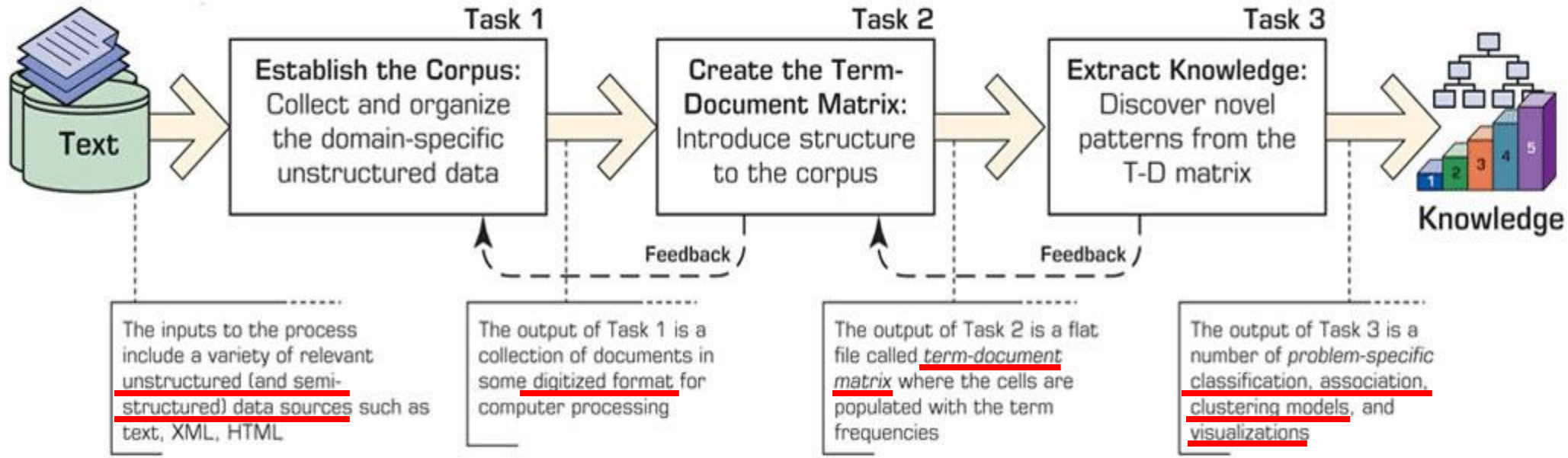
- (2) How can we reduce the dimensionality of this matrix to a manageable size because the TDM is often very large and rather sparse (most of the cells filled with zeros)? Several options are available:
  - **A domain expert** goes through the list of terms and eliminates those that do not make much sense for the context of the study (this is a manual, labor-intensive process).
  - Eliminate terms with **very few occurrences** in very few documents.
  - Transform the matrix using **Singular Value Decomposition (SVD)**, similar to PCA.

TM Process – Task 3: Knowledge Extraction



- The main categories of knowledge extraction methods are classification, clustering, association, and trend analysis.
  - **Classification**: The task is to classify a given data instance into a predetermined set of categories (or classes).
  - **Clustering**: It is an unsupervised process whereby objects are classified into "natural" groups called clusters. In clustering, the problem is to group an unlabeled collection of objects (e.g., documents, customer comments, Web pages) into meaningful clusters ***without any prior knowledge***.

TM Process – Task 3: Knowledge Extraction

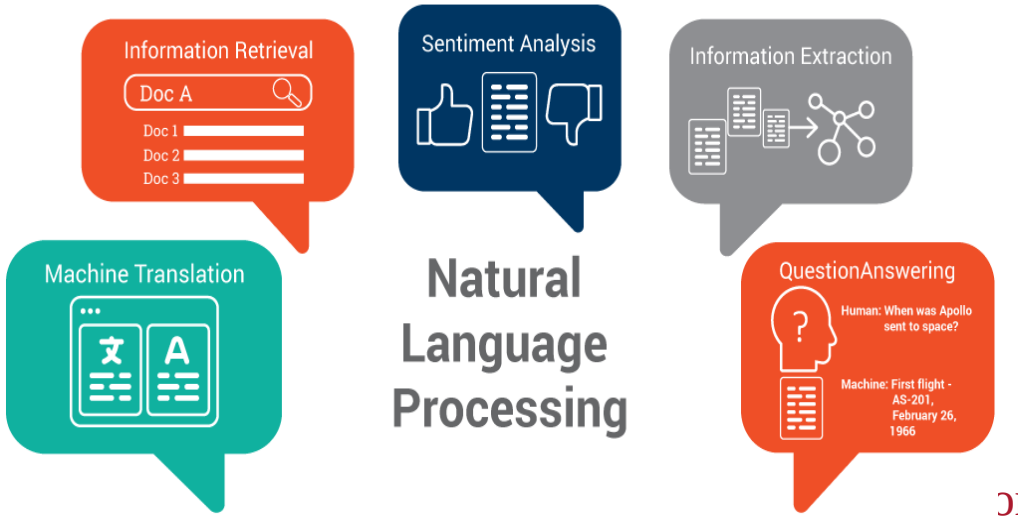


- The main categories of knowledge extraction methods are classification, clustering, association, and trend analysis.
  - **Association**: In text mining, associations specifically refer to the direct relationships between ***concepts (terms) or sets of concepts in documents***. "Enterprise Resource Planning" and "Customer Relationship Management" ➔ "Software Implementation Failure"
  - **Trend Analysis**: It is to identify ***the evolution of key concepts*** in the field of domains based upon a large number of articles.

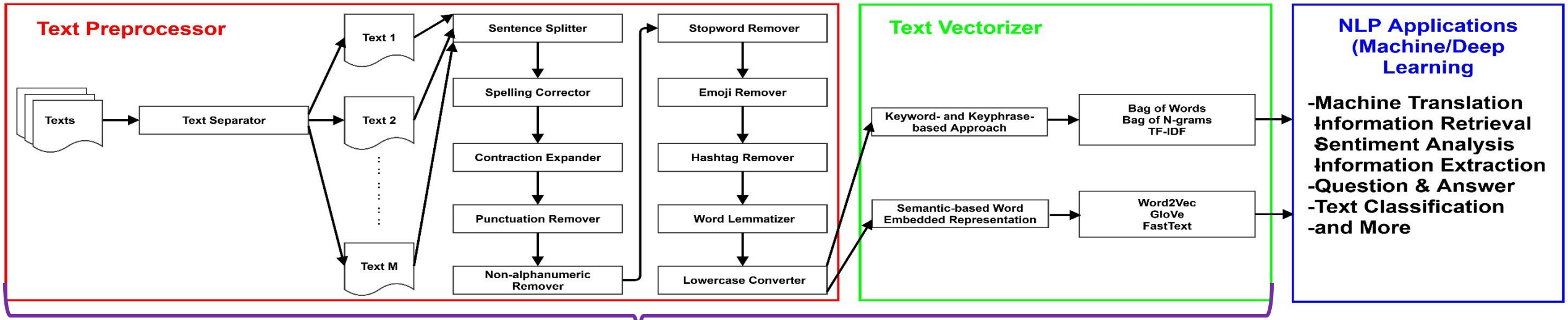
Natural Language Processing

- **Definition:** Natural Language Processing (NLP) is an area of research in computer science and artificial intelligence (AI) concerned with processing natural languages such as English or Chinese. This processing pipeline generally involves **translating natural language into numerical data** that a computer can understand and use in any domain application.

- **Some Applications:**
  - Machine Translation
  - Information Retrieval
  - Sentiment Analysis
  - Information Extraction
  - Question & Answer
  - Text Classification
  - And More



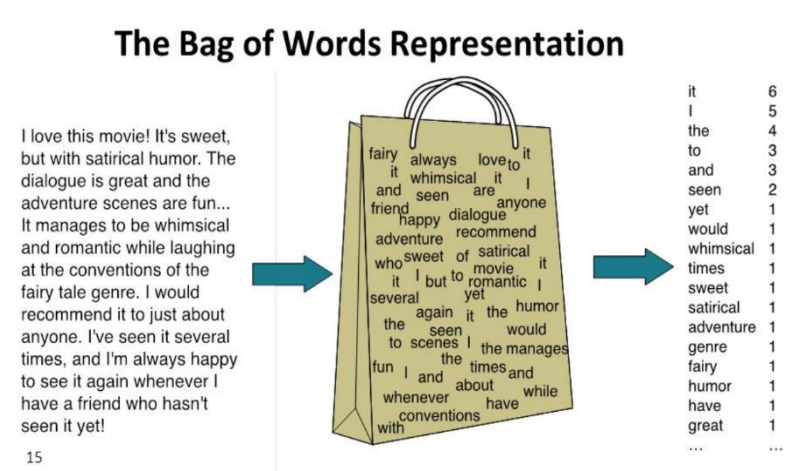
NLP Pipeline

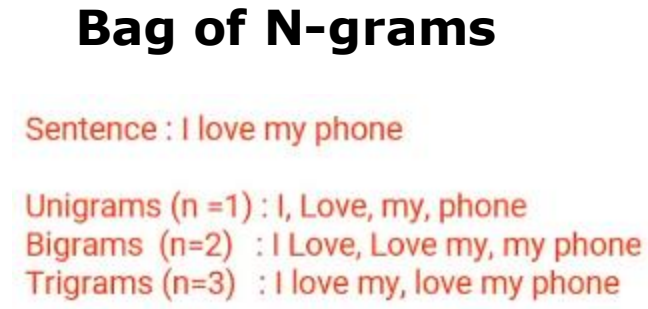


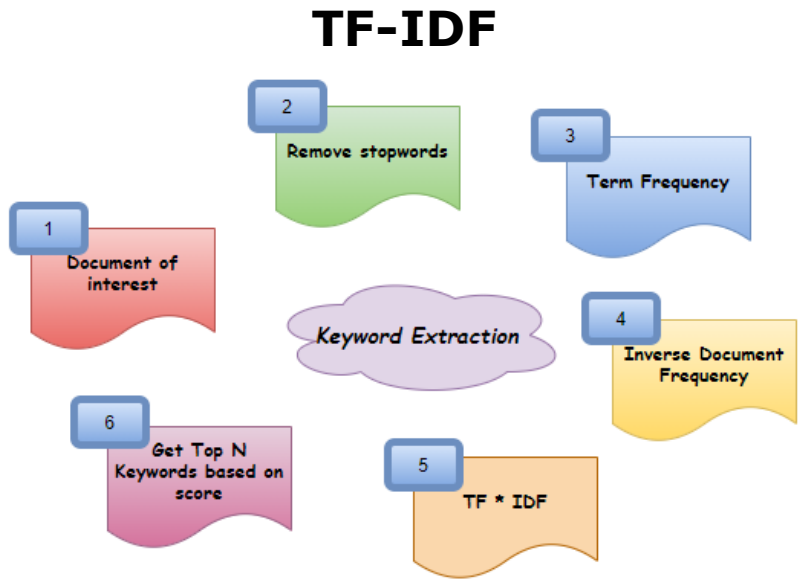
- **Text Separator (TS) module** segregates them from the corpus into many individual text.
- **Sentence Splitter (SS) module** splits each text into individual sentences.
- **Spelling Corrector (SC) module** conducts the spell check on each sentence and uses the unigram tokenization approach to replace misspelled words with the highest-probability corrected words.
- **Contraction Expander (CE) module** expands the contracted form of the words into a longer form, such as "I'm" to "I am", "You're" to "You are", "It's" to "It is", "S/He isn't" to "S/He is not", "They aren't" to "They are not" and "We aren't" to "We are not", in each sentence.
- **Punctuation Remover (PR)**, **Non-alphanumeric Remover (NR)**, **Stopword Remover (SR)**, **Emoji Remover (ER)**, and **Hashtag Remover (HR)** modules respectively remove punctuations (e.g., full stop(.), comma (,), and colon (:)), non-alphanumeric characters (e.g., #GoLangCode123!$! to GoLangCode123), stopwords (e.g., "ourselves", "hers", "between", "yourself", "but", and "again"), emojis (e.g., 😂, 😵, 😢, and 😱), and hashtags (e.g., #coffeelovers, #COVID-19, and #TheOscars) from expanded sentences.
- **Word Lemmatizer (WL) module** groups several forms of the same word together even if their spelling is quite different. For example, "walk", "walked", "walks", and "walking" would be treated the same as "walk". This extensive normalization down to the semantic root of a word is called **lemmatization**.
- **Lowercase Converter (LC) module** converts each character of a word into a lowercase on lemmatized sentences and then sends the processed sentences to Text Vectorizer.

NLP Pipeline – Text Vectorizer

- **Keyword- and Keyphrase-based Approach**
  - Bag of Words/ Bag of N-grams: Disregard grammar and even word/n-gram order but keeping multiplicity: The idea is to count ***the number of times each word/N-gram*** appears in each of the documents.
  - Term Frequency-Inverse Document Frequency (TF-IDF)







- **Semantic-based Word Embedded Representation**
  - Word2Vec (Google)
  - Glove (Standford)
  - FastText (Facebook)

Keyword- and Keyphrase-based Approach

T1 : *Without music life would be a mistake* 7 Words

T2 : *Radiohead are a great music band* 6 Words

- Bag of Words

without	music	life	would	be	a	mistake	radiohead	are	great	band
1	1	1	1	1	1	1	0	0	0	0
0	1	0	0	0	1	0	1	1	1	1

Keyword- and Keyphrase-based Approach

- Bag of Words

Let's recall the three types of movie reviews we saw earlier:

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

We will first build a vocabulary from all the unique words in the above three reviews. The vocabulary consists of these 11 words: 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'.

We can now take each of these words and mark their occurrence in the three movie reviews above with 1s and 0s. This will give us 3 vectors for 3 reviews:

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	2	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

Vector of Review 1: [1 1 1 1 1 1 1 0 0 0 0]

Vector of Review 2: [1 1 2 0 0 1 1 0 1 0 0]

Vector of Review 3: [1 1 1 0 0 0 1 0 0 1 1]

Keyword- and Keyphrase-based Approach

Exercise 1: Bag of Words

Given the following three simple documents:

- Doc 1 = “This is a short sentence”
- Doc 2 = “Yet another short sentence”
- Doc 3 = “Definitely a long sentence”

Please generate three document vectors (i.e., Doc 1, Doc 2, and Doc 3) by using Bag of Words.

Keyword- and Keyphrase-based Approach

T1 : *Without music life would be a mistake* 7 Words

T2 : *Radiohead are a great music band* 6 Words

- Bag of Bi-grams

without-music	music-life	life-would	would-be	be-a	a-mistake	radiohead-are	are-a	a-great	great-music	music-band
1	1	1	1	1	1	0	0	0	0	0
0	0	0	0	0	0	1	1	1	1	1

- Bag of Tri-grams

without-music-life	music-life-would	life-would-be	would-be-a	be-a-mistake	radiohead-are-a	are-a-great	a-great-music	great-music-band
1	1	1	1	1	0	0	0	0
0	0	0	0	0	1	1	1	1

Keyword- and Keyphrase-based Approach

Exercise 2: Bag of Bi-gram

Given the following three simple documents:

- Doc 1 = “This is a short sentence”
- Doc 2 = “Yet another short sentence”
- Doc 3 = “Definitely a long sentence”

Please generate three document vectors (i.e., Doc 1, Doc 2, and Doc 3) by using Bag of Bi-gram.

Keyword- and Keyphrase-based Approach

T1 : *Without music life would be a mistake* 7 Words

T2 : *Radiohead are a great music band* 6 Words

- TF-IDF

Term Frequency = (count of the term) / (total word count in the document)

Inverse Document Frequency = log (number of docs) / (docs containing keyword)

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

$tf_{i,j}$ = number of occurrences of i in j
df_i = number of documents containing i
N = total number of documents

without	music	life	would	be	a	mistake	radiohead	are	great	band
0.043	0	0.043	0.043	0.043	0	0.043	0	0	0	0
0	0	0	0	0	0	0	0.05	0.05	0.05	0.05

Keyword- and Keyphrase-based Approach

Let's recall the three types of movie reviews we saw earlier:

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

- TF-IDF

1

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	2	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

2

Term	Review 1	Review 2	Review 3	TF (Review 1)	TF (Review 2)	TF (Review 3)
This	1	1	1	1/7	1/8	1/6
movie	1	1	1	1/7	1/8	1/6
is	1	2	1	1/7	1/4	1/6
very	1	0	0	1/7	0	0
scary	1	1	0	1/7	1/8	0
and	1	1	1	1/7	1/8	1/6
long	1	0	0	1/7	0	0
not	0	1	0	0	1/8	0
slow	0	1	0	0	1/8	0
spooky	0	0	1	0	0	1/6
good	0	0	1	0	0	1/6

3

Term	Review 1	Review 2	Review 3	IDF
This	1	1	1	0.00
movie	1	1	1	0.00
is	1	2	1	0.00
very	1	0	0	0.48
scary	1	1	0	0.18
and	1	1	1	0.00
long	1	0	0	0.48
not	0	1	0	0.48
slow	0	1	0	0.48
spooky	0	0	1	0.48
good	0	0	1	0.48

4

Term	Review 1	Review 2	Review 3	IDF	TF-IDF (Review 1)	TF-IDF (Review 2)	TF-IDF (Review 3)
This	1	1	1	0.00	0.000	0.000	0.000
movie	1	1	1	0.00	0.000	0.000	0.000
is	1	2	1	0.00	0.000	0.000	0.000
very	1	0	0	0.48	0.068	0.000	0.000
scary	1	1	0	0.18	0.025	0.022	0.000
and	1	1	1	0.00	0.000	0.000	0.000
long	1	0	0	0.48	0.068	0.000	0.000
not	0	1	0	0.48	0.000	0.060	0.000
slow	0	1	0	0.48	0.000	0.060	0.000
spooky	0	0	1	0.48	0.000	0.000	0.080
good	0	0	1	0.48	0.000	0.000	0.080

Keyword- and Keyphrase-based Approach

Exercise 3: TF-IDF

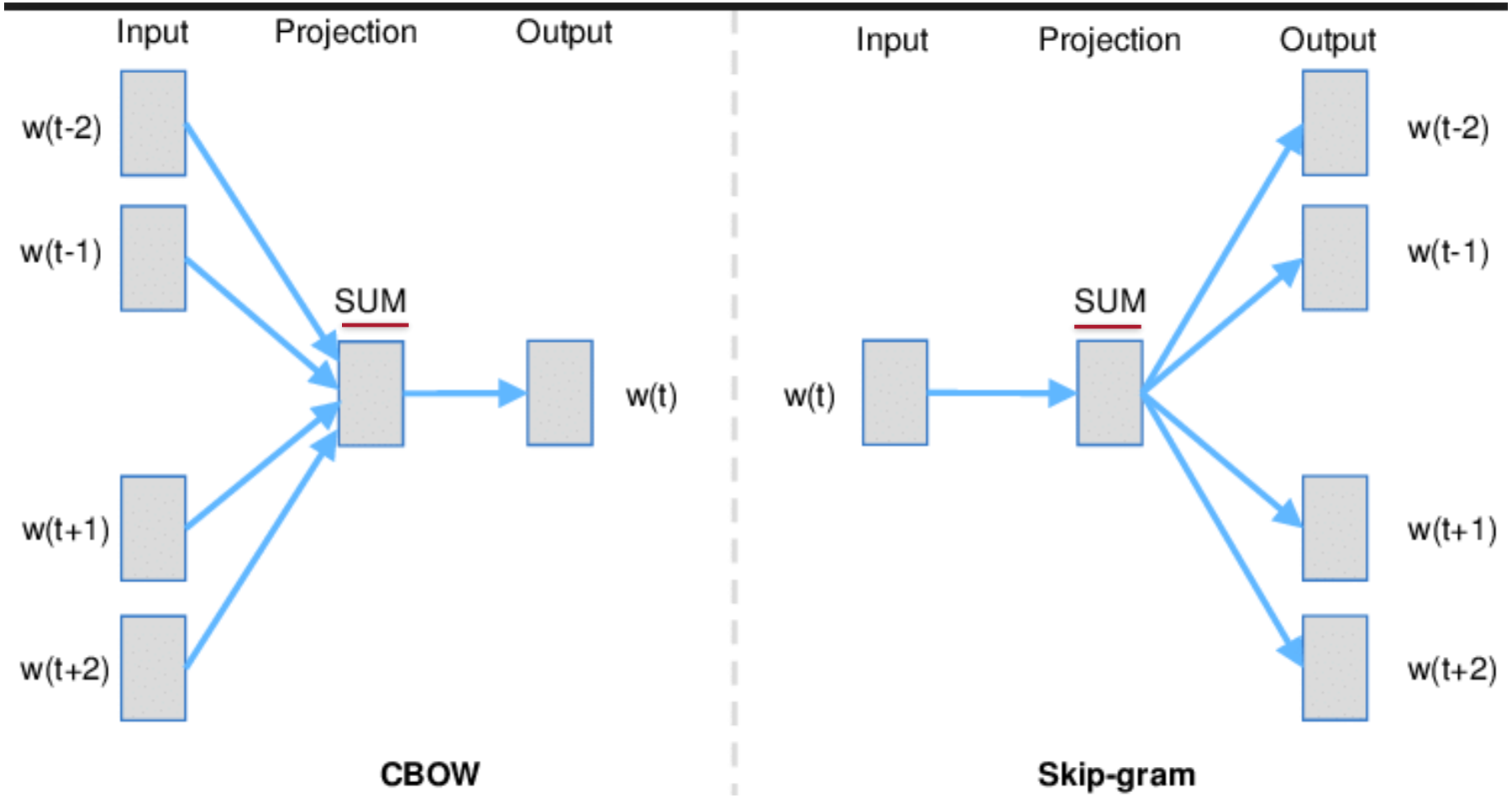
Given the following three simple documents:

- Doc 1 = “This is a short sentence”
- Doc 2 = “Yet another short sentence”
- Doc 3 = “Definitely a long sentence”

Please generate three document vectors (i.e., Doc 1, Doc 2, and Doc 3) by using TF-IDF

Semantic-based Word Embedded Representation – Word2Vec

- Continuous Bag of Words (CBoWs) and Continuous Skip-grams



Embedding means representing or coding an object or a pattern in a meaningful format, and this format is a vector.

The **CBOW** model learns word embedding by predicting the current word based on the context.

The **Skip-gram** model learns embedding by predicting context words given the current word.

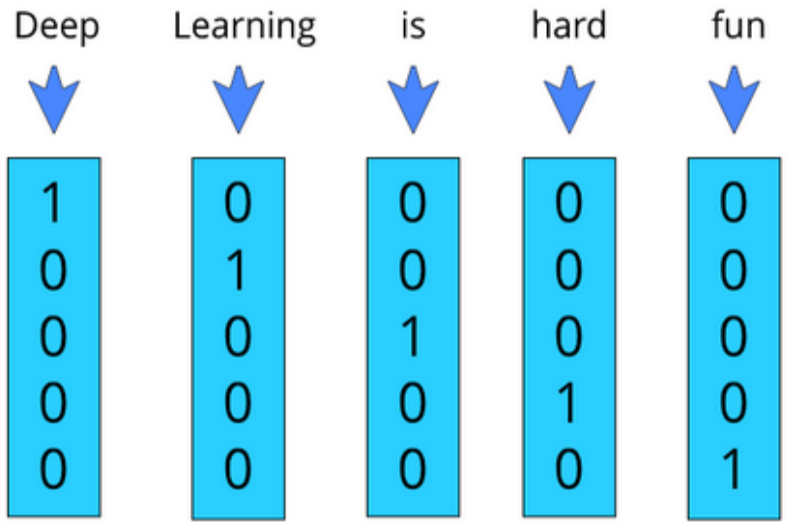
Both focus on learning words based on **local context**, which context is defined by the neighborhood word window.

The **window size w** is a configurable parameter of the word embedding model.

Word2Vec – One-Hot Encoding

- Represent each word as a **vector of length V**, where **V is the total number of unique words available in the entire textual data.**

- V is also called the **vocabulary count**. **Each word's one hot encoded vector is essentially a binary vector with the value 1 being in a *unique index* for each word** and the value 0 being in every other index of the vector. Let's visualize this.



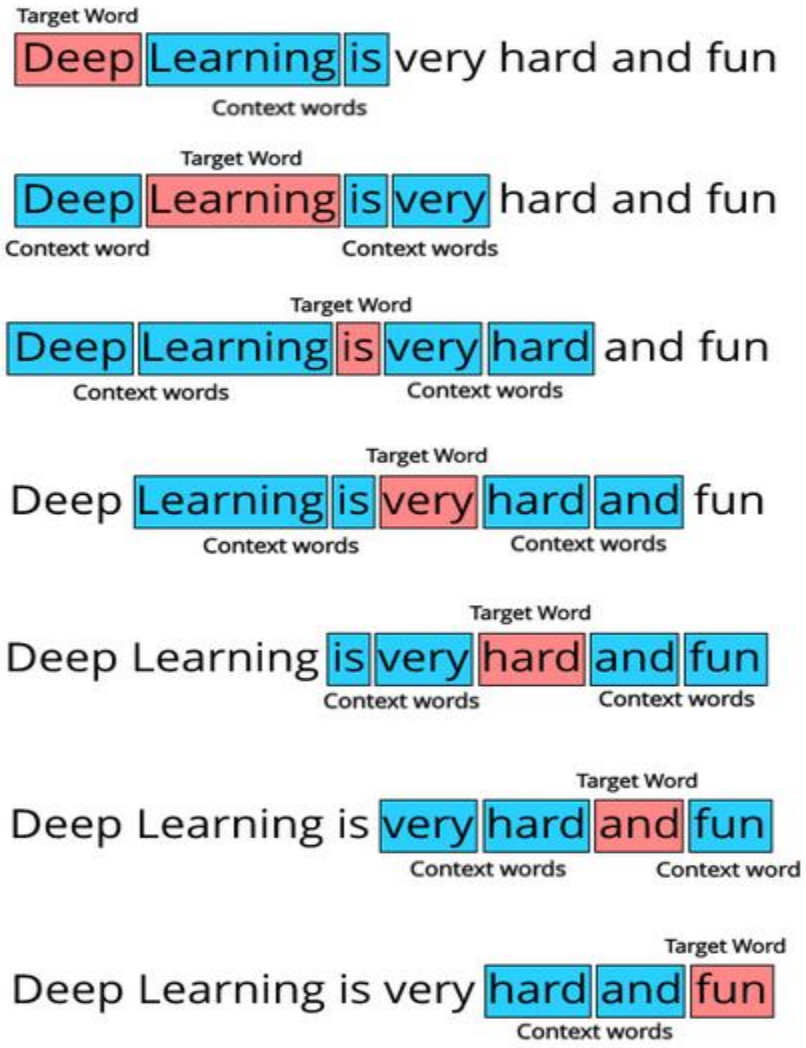
- Suppose our data comprises of the following 2 texts:
  - T1: Deep learning is hard
  - T2: Deep learning is fun

- The value of **V is 5**, because there are 5 unique words: ['Deep', 'learning', 'is', 'hard', 'fun']. Now to represent each word, we will use **a vector of length 5**.

- **The size of each word's one hot encoded vector will always be V**, which is the **size of the entire vocabulary**. ***In the example, it was 5 but usually the value of V can reach 10k or even 100k***.

  ➔ *A **huge vector** represent a **single word**.*

Word2Vec – Skip-grams



1st Window pairs: (Deep, Learning), (Deep, is)

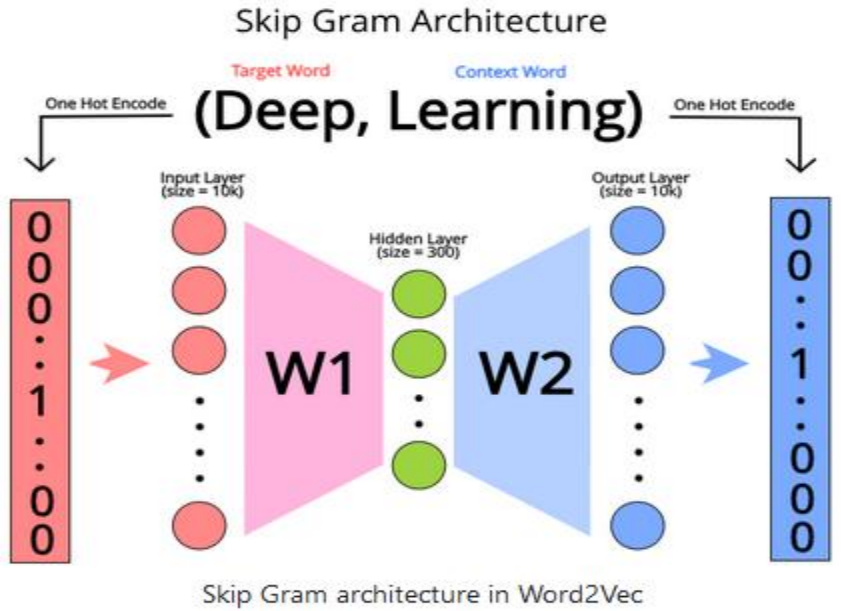
2nd Window pairs: (Learning, Deep), (Learning, is), (Learning, very)

3rd Window pairs: (is, Deep), (is, Learning), (is, very), (is, hard)

And so on. At the end our target word vs context word data set is going to look like this:

(Deep, Learning), (Deep, is), (Learning, Deep), (Learning, is), (Learning, very), (is, Deep), (is, Learning), (is, very), (is, hard), (very, learning), (very, is), (very, hard), (very, and), (hard, is), (hard, very), (hard, and), (hard, fun), (and, very), (and, hard), (and, fun), (fun, hard), (fun, and)

This can be considered as our *"training data"* for word2vec.



Skip Gram architecture in Word2Vec

- The neural network trying to do here is to **guess which context words can appear given a target word**.
- **Input** – "Deep" word vector
- **Output:**
  - "Learning" word vector
  - "is" word vector
- After training the neural network, if we input any target word into the neural network, it will give a vector output which represents the word which has **a high probability** of appearing near the given word.

Word2Vec – Skip-grams

- The skip-gram approach works well with small text corpora and rare terms.

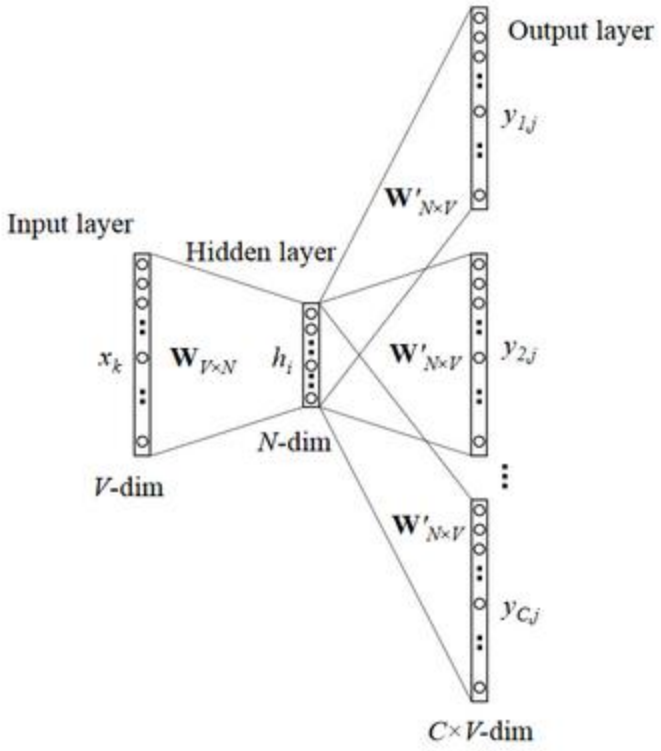


Figure 3: The skip-gram model

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…

Input	Output
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

Word2Vec – Skip-grams

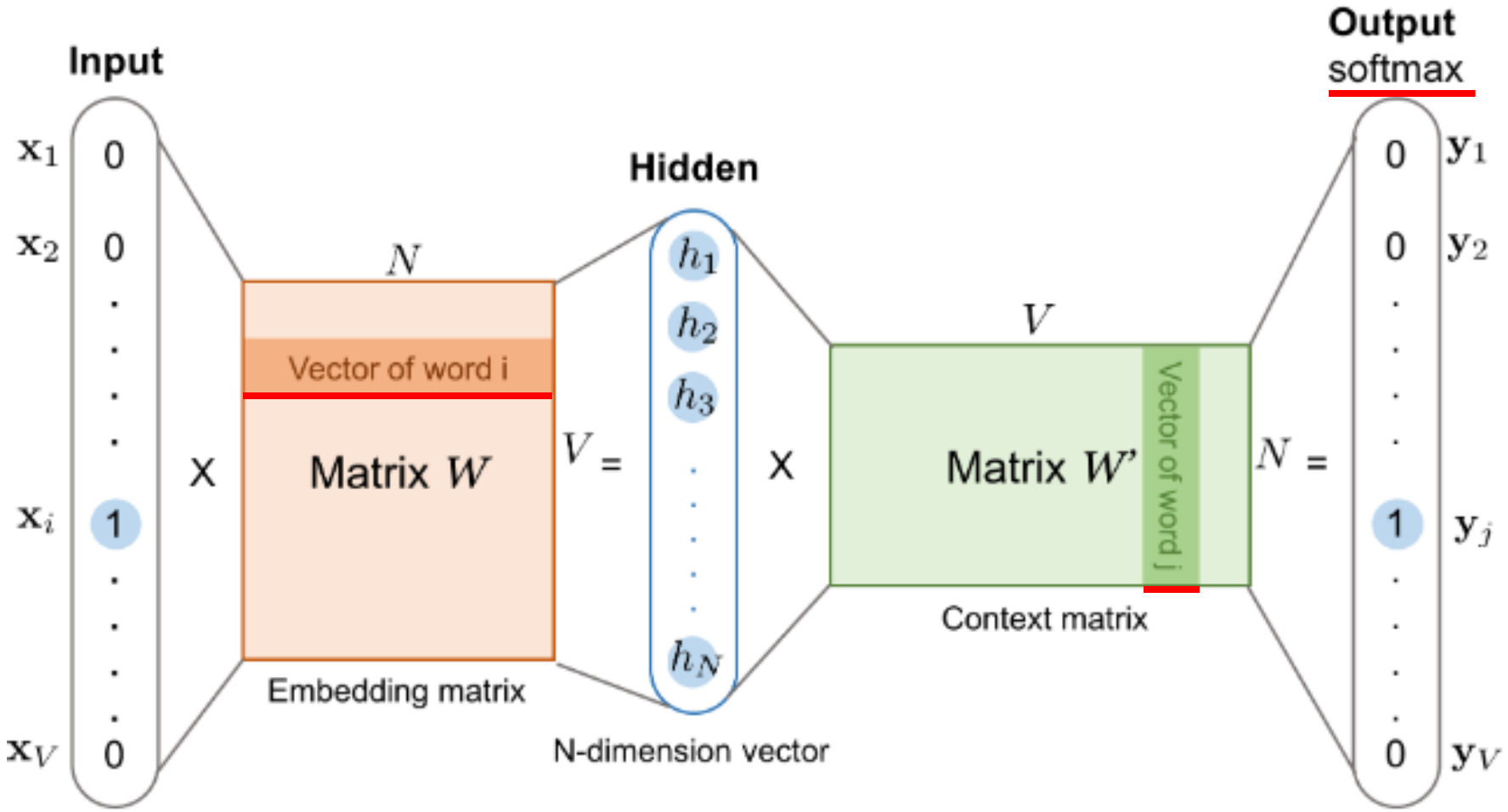
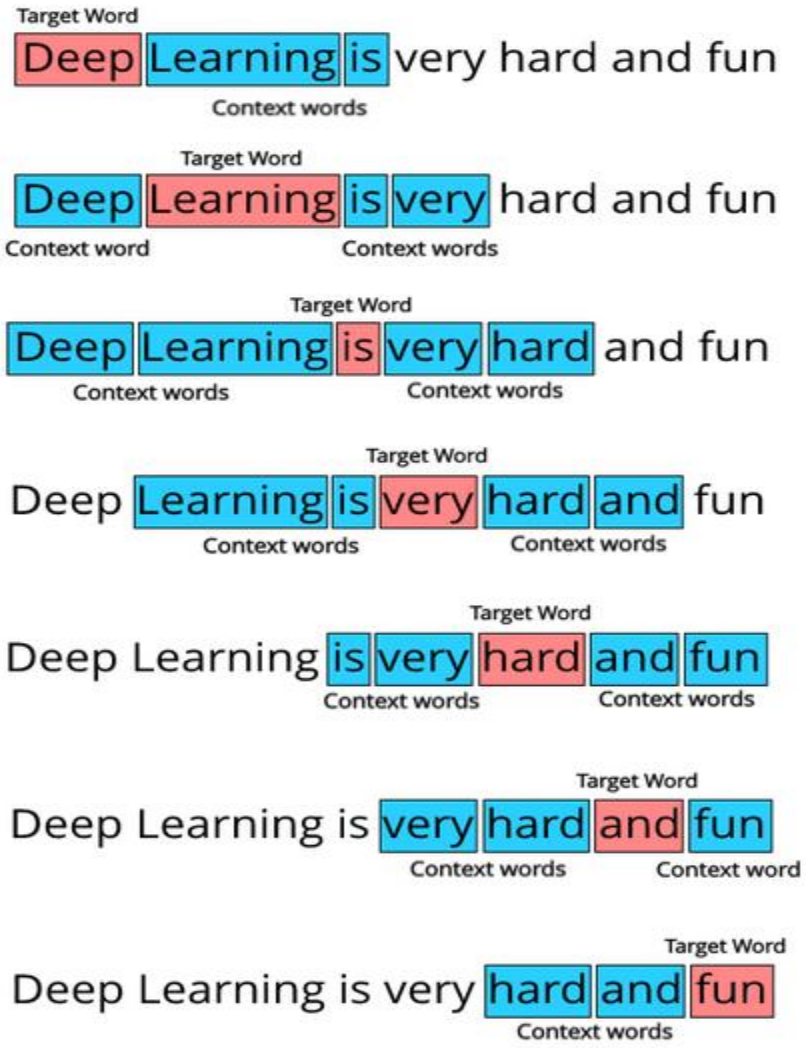


Fig. 1. The skip-gram model. Both the input vector $\mathbf{x}$ and the output $\mathbf{y}$ are one-hot encoded word representations. The hidden layer is the word embedding of size N.

Word2Vec – CBoWs



1st Window pairs: (Deep, Learning), (Deep, is)

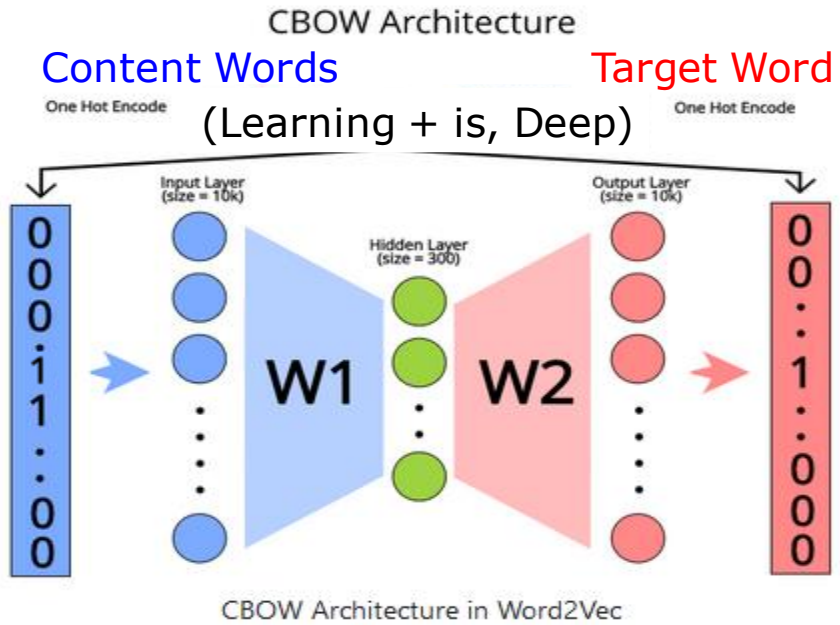
2nd Window pairs: (Learning, Deep), (Learning, is), (Learning, very)

3rd Window pairs: (is, Deep), (is, Learning), (is, very), (is, hard)

And so on. At the end our target word vs context word data set is going to look like this:

(Deep, Learning), (Deep, is), (Learning, Deep), (Learning, is), (Learning, very), (is, Deep), (is, Learning), (is, very), (is, hard), (very, learning), (very, is), (very, hard), (very, and), (hard, is), (hard, very), (hard, and), (hard, fun), (and, very), (and, hard), (and, fun), (fun, hard), (fun, and)

This can be considered as our *"training data"* for word2vec.



CBOW Architecture in Word2Vec

- For CBOW, we try to **predict the target word given the context words**.
- For example, if we give the sentence: _____ Learning is very hard and fund, where **["Learning", "is"] represents the context words**, the neural network should give **"Deep" as the output target word**. This is the core task the neural network tries to train for in the case of CBOW.
- **Input** – "Learning" word vector + "is" word vector
- **Output** – "Deep" word vector

Word2Vec – CBoWs

- The CBoW approach shows higher accuracy for frequent words and is much faster to train.

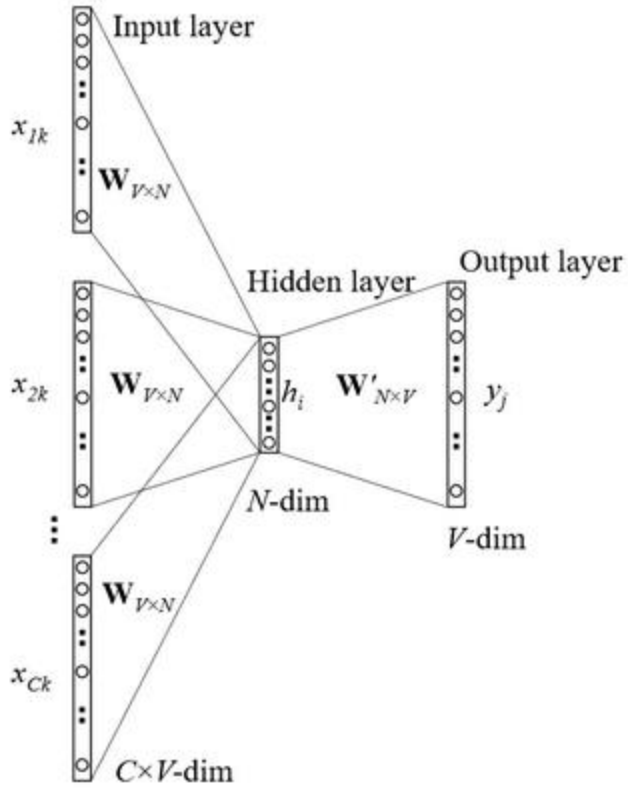


Figure 2: Continuous bag-of-word model

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…
thou	shalt	not	make	a	machine	in	the	…

Output	Input
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

Word2Vec – CBoWs

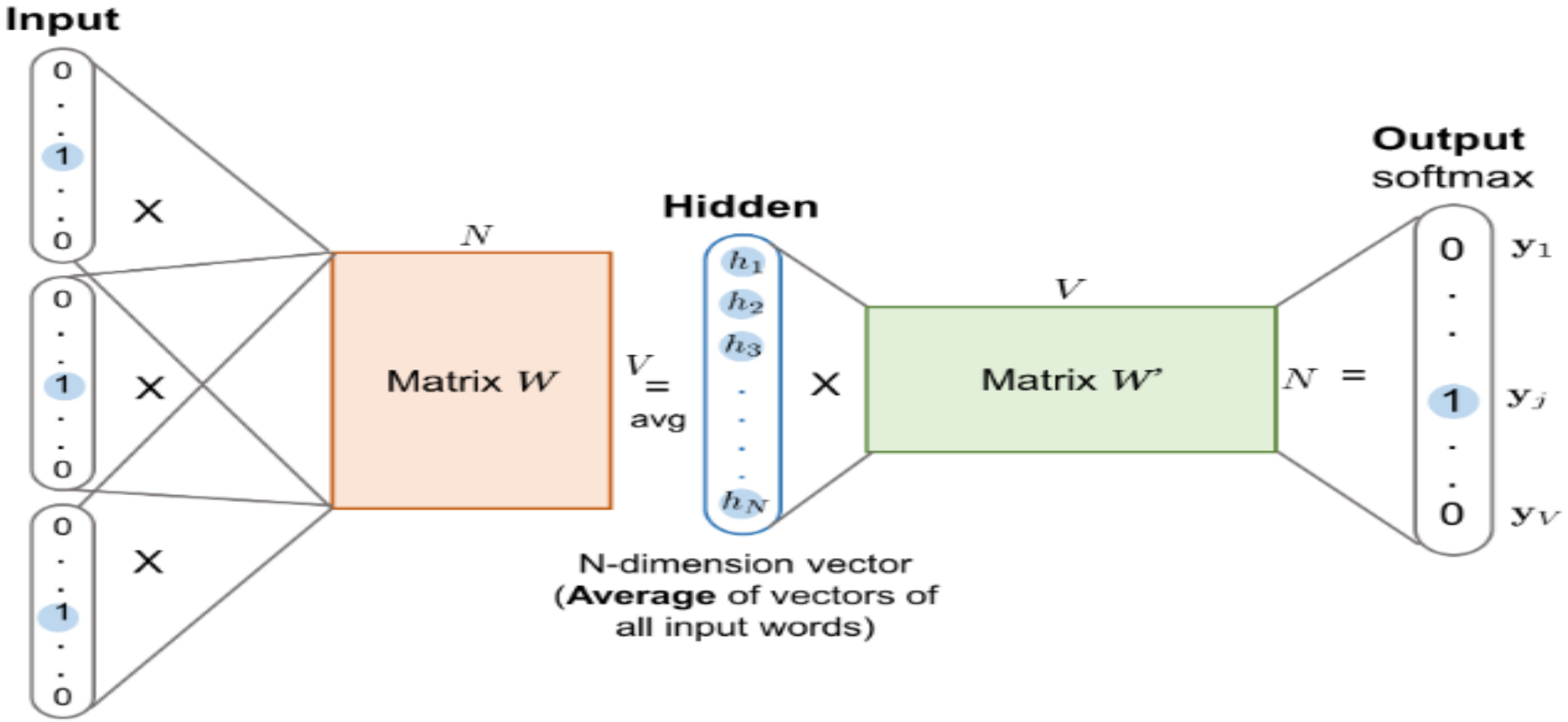
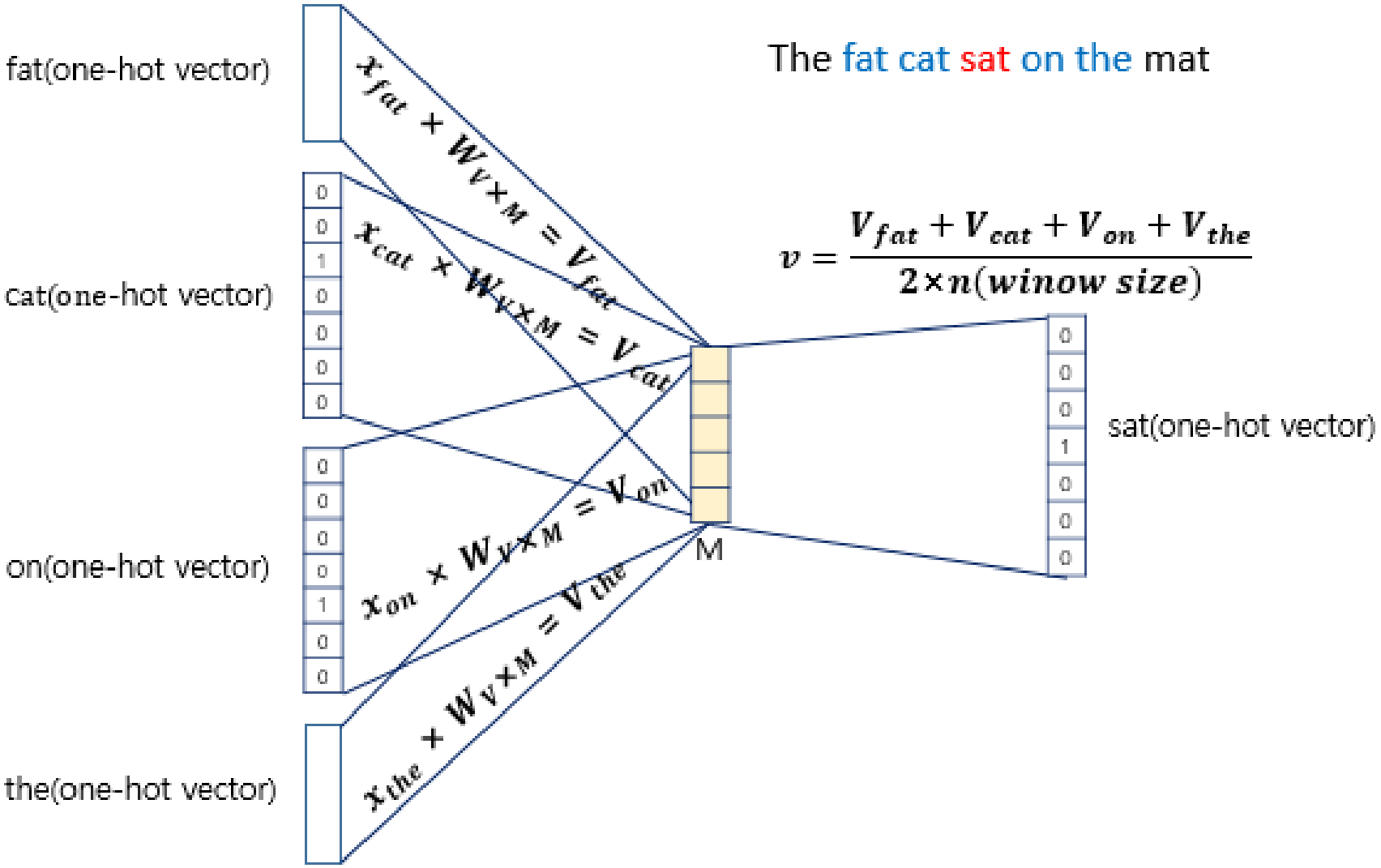


Fig. 2. The CBOW model. Word vectors of multiple context words are averaged to get a fixed-length vector as in the hidden layer. Other symbols have the same meanings as in Fig 1.

stitute

Word2Vec – CBoWs



Hands-on Example: CBoW Model

```python
import torch
import torch.nn as nn
word_to_ix = {}

# Class to build CBOW model for developing word embeddings.
class CBOW(torch.nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super(CBOW, self).__init__()

        # out: 1 x emdedding_dim
        # embedding_dim (int) - the size of each embedding vector, e.g., 100, for each vocab
        # Embedding(49, 100) Without Bias
        self.embeddings = nn.Embedding(vocab_size, embedding_dim)
        # in_features (int) - size of each input sample, e.g., 100
        # out_features (int) - size of each output sample, 128
        # Linear(in_features=100, out_features=128, bias=True)
        self.linear1 = nn.Linear(embedding_dim, 128) # Default: bias = True
        self.activation_function1 = nn.ReLU()

        # out: 1 x vocab_size
        # Linear(in_features=128, out_features=49, bias=True)
        self.linear2 = nn.Linear(128, vocab_size)
        self.activation_function2 = nn.LogSoftmax(dim=-1)

    def forward(self, inputs):
        embeds = sum(self.embeddings(inputs)).view(1, -1)
        out = self.linear1(embeds)
        out = self.activation_function1(out)
        out = self.linear2(out)
        out = self.activation_function2(out)
        return out

    def get_word_emdedding(self, word):
        word = torch.tensor([word_to_ix[word]])
        return self.embeddings(word).view(1, -1)
```

```python
# Let us define the vector in which the embedding will be stored.
def make_context_vector(context, word_to_ix):
    idxs = [word_to_ix[w] for w in context]
    return torch.tensor(idxs, dtype=torch.long)
```

Hands-on Example: CBoW Model

```python
def main():
    CONTEXT_SIZE = 2  # 2 words to the left, 2 to the right
    EMDEDDING_DIM = 100

    # Here we use a small paragraph as raw text to train a word embedding.
    raw_text = """We are about to study the idea of a computational process.
    Computational processes are abstract beings that inhabit computers.
    As they evolve, processes manipulate other abstract things called data.
    The evolution of a process is directed by a pattern of rules
    called a program. People create programs to direct processes. In effect,
    we conjure the spirits of the computer with our spells.""".split()

    # First, we need to build a vocabulary!
    # There are 49 vocabularies
    vocab = set(raw_text)
    vocab_size = len(vocab)

    global word_to_ix
    # Create a dictionary, vocab is a key, and the index is the value from 0 to 48.
    word_to_ix = {word: ix for ix, word in enumerate(vocab)}
    # Create a dictionary, index is a key from 0 to 48, and the vocab is the value.
    ix_to_word = {ix: word for ix, word in enumerate(vocab)}

    # Create our dataset using a combination of context and target
    data = []
    for i in range(CONTEXT_SIZE, len(raw_text) - CONTEXT_SIZE):
        # Two words on the left and Two words on the right
        context = [raw_text[i - 2], raw_text[i - 1],
                   raw_text[i + 1], raw_text[i + 2]]
        # The word in the middle between them
        target = raw_text[i]
        # Append context and target into the data list for training.
        data.append((context, target))

    # Next, we will create the model using the CBOW class:
    model = CBOW(vocab_size, EMDEDDING_DIM)
    loss_function = nn.NLLLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.001)

    # Now we are ready to do Training
    for epoch in range(50):
        total_loss = 0
        for context, target in data:
            context_vector = make_context_vector(context, word_to_ix)
            log_probs = model(context_vector)
            total_loss += loss_function(log_probs, torch.tensor([word_to_ix[target]]))

        # Optimize at the end of each epoch
        optimizer.zero_grad()
        total_loss.backward()
        optimizer.step()

    # And Then Testing
    context = ['People', 'create', 'to', 'direct']
    context_vector = make_context_vector(context, word_to_ix)
    a = model(context_vector)

    # Print result: The CBOW model uses context words ("People", "create", "to", "direct")
    # to predict the target word programs. We could get the 100-dimensional word embedding from the model
    # , where EMDEDDING_DIM = 100.
    print()
    print(f'Raw text: {" ".join(raw_text)}\n')
    print(f'Context: {context}\n')
    print(f'Prediction: {ix_to_word[torch.argmax(a[0]).item()]}')
    print()
    print(f'embedding:{model.get_word_emdedding("programs")}')

if __name__ == "__main__":
    main()
```